

CSC 330: Assignment 4

Deadline: Sun, Mar 22 at 23.59

Submit via: [CSC 330 Submission Portal](#)

Ground rules

- You must complete this assignment *entirely* on your own. In other words, you should come up with the solution *yourself*, write the code *yourself*, and test the code *yourself*. The university policies on academic dishonesty (a.k.a. cheating) will be taken very seriously. Check the syllabus for details on what is allowed and what is not.
 - You can have high-level discussions with your classmates about the course material.
 - You are also welcome to use Teams or come to office hours and ask questions. If in doubt, please ask—we are here to help!
 - Using LLMs to get the solutions is akin to shooting yourself in the foot: you *might* pass the assignment, but you will most likely fail the written exams.
- You can use a maximum of **three (3) late days per assignment**, and a maximum of **seven (7) days for the whole semester**. Any further accommodations must be announced and negotiated with the instructor at least seven (7) days before the deadline.
- You will need to gather at least half of the total assignment score to pass the course.
- If you think some things should be explained in detail, please include the explanations in the form of source code comments.
- We will try to grade your assignments within seven (7) days of the initial submission deadline.
- If you think there is a problem with your grade, you have one week to raise concerns after the grades go public. TAs will be holding office hours during those seven days to address any such problems. After that, your grade is set in stone.
- Document your code where needed!
- Watch your style! We will penalize submissions with inadequate or bad style.
- Your programs should generate **no** errors or warnings. If a program generates a single compilation error or warning, it will receive a zero.
- Please indicate your V-number and name within a comment at the top of each submitted file.
 - **Do not** include other private information (full name, your address, telephone number, medical exam tests etc.).

Furthermore, for this assignment:

- Use the provided templates and tests in the attached `code.zip`. Replace `TODO` parts with your code. Leave the rest alone.
- You should only submit a single ZIP file containing two a single OCaml and a single Ruby file to the CSC 330 Submission Portal. The contents of the ZIP must include `prob1.rb` and `prob1.ml` (lowercase).
 - Warning: different file names might result in zero points. Our scripts expect the correct file names.
 - Another warning: put the filenames at the root of the ZIP file; if you put them within a directory, our script might not be able to find it.
- Try not to stray away from the features discussed in class.

Test examples, updates and error corrections will be available on Teams and the CSC 330 Submission Portal. Make sure to check it from time to time.

Testing

Use the provided files (`prob1_test.ml` and `prob1_test.rb`) for sample test cases. The tests are organized in such a way that they will help you complete your assignment.

Problem 1: Geometry [50 points]

For this assignment, you will complete and extend two implementations of an interpreter for a small “language” for two-dimensional geometry objects. An implementation in OCaml is mostly completed for you. An implementation in Ruby is mostly not completed. The OCaml implementation is structured with functions and pattern-matching. The Ruby implementation is structured with subclasses and methods, including some mind-bending double dispatch and other dynamic dispatch to stick with an OOP style. You will decide later on which one do you like more and why.

Start with `prob1.ml` and `prob1.rb` located within the assignment package. Some example tests are also provided.

Language Semantics

Our “language” has five kinds of values and four other kinds of expressions. The representation of expressions depends on the metalanguage (OCaml or Ruby), with this same semantics:

- A **Nope** represents the empty set of two-dimensional points.
- A **Point** represents a two-dimensional point with an x -coordinate and a y -coordinate. Both coordinates are floating-point numbers.
- A **Line** is a non-vertical infinite line in the plane, represented by a slope and an intercept (as in $y = mx + b$ where m is the slope and b is the intercept), both floating-point numbers.
- A **VerticalLine** is an infinite vertical line in the plane, represented by its x -coordinate.
- A **LineSegment** is a (finite) line segment, represented by the x - and y -coordinates of its endpoints (so four total floating-point numbers).
- An **Intersect** expression is not a value. It has two subexpressions. The semantics is to evaluate the subexpressions (in the same environment) and then return the value that is the intersection (in the geometric sense) of the two subresults. For example, the intersection of two lines could be one of:
 - **Nope**, if the lines are parallel
 - a **Point**, if the lines intersect
 - a **Line**, if the lines have the same slope and intercept (see the note below about what *the same* means for floating-point numbers)
- A **Let** expression is not a value. It is like let-expressions in other languages we have studied: the first subexpression is evaluated and the result bound to a variable that is added to the environment for evaluating the second subexpression.
- A **Var** expression is not a value. It is for using variables in the environment: we look up a string in the environment to get a geometric value.
- A **Shift** expression is not a value. It has a dX (a floating-point number), a dY (a floating-point number), and a subexpression. The semantics is to evaluate the subexpression and then shift the result by dX (in the x -direction; positive is *to the right*) and dY (in the y -direction; positive is *up*). More specifically, shifting for each form of value is as follows:
 - **Nope** remains **Nope**.
 - A **Point** representing (x, y) becomes a **Point** representing $(x + dX, y + dY)$.
 - A **Line** with slope m and intercept b becomes a **Line** with slope m and an intercept of $b + dY - m \cdot dX$.
 - A **VerticalLine** becomes a **VerticalLine** shifted by dX ; the dY is irrelevant.
 - A **LineSegment** has its endpoints shift by dX and dY .

Note on Floating-Point Numbers

Because arithmetic with floating-point numbers can introduce small rounding errors, it is rarely appropriate to use equality to decide if two floating-point numbers are the same. Instead, the provided code uses a helper function/method to decide if two floating-point numbers are “real close” (for our purposes, within 0.00001) and all your code should follow this approach as needed. For example, two points are the same if their x -coordinates are within 0.00001 and their y -coordinates are within 0.00001.

Note that this method is not bullet-proof either; [the more robust solution requires deep understanding of how floating point numbers work](#). While the previous link is a long read, consider reading it at least once in your career.

Expression Preprocessing

To simplify the interpreter, we first preprocess expressions. Preprocessing takes an expression and produces a new, equivalent expression with the following invariants:

- No **LineSegment** anywhere in the expression has endpoints that are the same as (i.e., float-close to) each other. Such a line-segment should be replaced with the appropriate **Point**. For example in OCaml syntax, `LineSegment (3.2, 4.1, 3.2, 4.1)` should be replaced with `Point (3.2, 4.1)`.

- Every `LineSegment` has its first endpoint (the first two float values in OCaml) to the right (higher x -value) of the second endpoint. If the x -coordinates of the two endpoints are the same (float-close), then the `LineSegment` has its first endpoint above (higher y -value) the second endpoint. For any `LineSegment` not meeting this requirement, replace it with a `LineSegment` with the same endpoints reordered. Maintain this convention in both the OCaml and the Ruby code.

Admittedly this order can seem *backwards* especially when reading the OCaml code but sometimes reading code involves accepting a convention decided by someone else.

The OCaml Code

Most of the OCaml solution is given to you! All you have to do is add preprocessing (problem 1) and `Shift` expressions (problem 2). The sample solution around 20 lines of pretty boilerplate code. As always, line counts are just a rough guide.

Notice the OCaml code is organized around a variant definition for expressions, functions for the different operations, and pattern-matching to identify different cases. The interpreter `eval_prog` uses a helper function `intersect` with cases for every combination of geometric value (so with 5 kinds of values there are 25 cases though some are handled together via pattern-matching). The surprisingly complicated part is the algorithm for intersecting two line segments.

The Ruby Code

Much of the Ruby solution is not given to you! To get you started in the desired way, we have defined classes for each kind of expression in our language, as well as appropriate superclasses. We have implemented parts of each class and left comments with what you need to do to complete the implementation as described in more detail in problems 3 and 4. The sample solution added about 200 lines of Ruby code, many of which were `end`. As always, line counts are just a rough guide.

Notice the Ruby code is organized around classes where each class has methods for various operations. All kinds of expressions need methods for preprocessing and evaluation. They are subclasses of `GeometryExpression` just like all OCaml constructors are part of the `geom_exp` variant type (though the `GeometryExpression` class turns out not to be so useful). The value subclasses also need methods for shifting and intersection and they subclass `GeometryValue` so that some shared methods can be inherited (in analogy with some uses of wildcard patterns and helper functions in OCaml).

Your Ruby code should follow these general guidelines:

- All your geometry-expression objects should be **immutable**: assign to instance variables only when constructing an object. To *change* a field, create a new object.
- The geometry-expression objects have public getter methods: like in the OCaml code, the entire program can assume the expressions have various coordinates, subexpressions, etc.
- Unlike in OCaml, you do not need to define exceptions since without a type-checker we can just *assume* the right objects are used in the right places. You can also use `raise` with just a string as appropriate.
- Follow OOP-style. In particular, operations should be instance methods and you **should not use methods** like `is_a?`, `instance_of?`, `class`, etc. This makes problem 4 much more difficult, which is the purpose of the problem.

Advice for Approaching the Assignment

- Understand the high-level structure of the code and how the OCaml and Ruby files are structured in different ways before diving into the details.
- **Approach the questions in order even though there is some flexibility (e.g., it is possible to do the Ruby problems before the OCaml problems).**
- Because almost all the OCaml code is given to you, for much of the Ruby implementation, you can port the corresponding part of the OCaml solution. Doing so makes your job **much** easier (e.g., you need not re-figure out facts about geometry). Porting existing code to a new language is a useful and realistic skill to develop. It also helps teach the similarities and differences between languages.
- Be sure to test each line of your Ruby code. Dynamically typed languages require testing things that other languages catch for you statically. Ruby will not even tell you statically if you misspell a method name or instance variable.

The Problems:

1. [5 points] Implement an OCaml function `preprocess_prog` of type `geom_exp → geom_exp` to implement expression preprocessing as defined above. The idea is that evaluating program `e` would be done with `eval_prog (preprocess_prog e) []` where the `[]` is the empty list for the empty environment.
2. [5 points] Add shift expressions support (as defined above) to the OCaml implementation by adding the constructor `Shift of float * float * geom_exp` to the definition of `geom_exp` and adding appropriate branches to `eval_prog` and `preprocess_prog`. The first float is dX and the second is dY . Do not change other functions. In particular, there is no

need to change `intersect` because this function is used only for values in our geometry language and shift expressions are not geometry values.

3. [20 points] Complete the Ruby implementation except for intersection, which means skip for now additions to the `Intersect` class and, more importantly, methods related to intersection in other classes. Do not modify the code given to you. Follow this approach:

- Every subclass of `GeometryExpression` should have a `preprocess_prog` method that takes no arguments and returns the geometry object that is the result of preprocessing `self`. To avoid mutation, return a new instance of the same class unless it is trivial to determine that `self` is already an appropriate result.
- Every subclass of `GeometryExpression` should have an `eval_prog` method that takes one argument, the environment, which you should represent as an array whose elements are two-element arrays: a Ruby string (the variable name) in index 0 and an object that is a value in our language in index 1. As in any interpreter, pass the appropriate environment when evaluating subexpressions (fairly easy since we do not have closures this time). To make sure you handle both scope and shadowing correctly:
 - Do not ever mutate an environment; create a new environment as needed instead. Be careful what methods you use on arrays to avoid mutation.
 - The `eval_prog` method in `Var` is given to you. Make sure the environments you create work correctly with this definition.

The result of `eval_prog` is the result of *evaluating the expression represented by self*, so, as we expect with OOP style, the cases of OCaml's `eval_prog` are spread among our classes, just like with `preprocess_prog`.

- Every subclass of `GeometryValue` should have a `shift` method that takes two arguments `dx` and `dy` and returns the result of shifting `self` by `dx` and `dy`. In other words, all values in the language *know how to shift themselves to create new objects*. Hence the `eval_prog` method in the `Shift` class should be very short.
 - Remember you should not use any method like `is_a?`, `instance_of?`, `class`, etc.
 - Analogous to OCaml, an overall program `e` would be evaluated via `e.preprocess_prog.eval_prog []` (note our use of an array for the environment).
4. [20 points] Implement intersection in your Ruby solution following the directions here, in which we require both double dispatch and a separate use of dynamic dispatch for the line-segment case. Remember all the different cases in OCaml will appear somewhere in the Ruby solution, just arranged very differently.
- Implement `preprocess_prog` and `eval_prog` in the `Intersect` class. This is not difficult, much like your prior work in the `Shift` class is not difficult. This is because every subclass of `GeometryValue` will have an `intersect` method that *knows how to intersect itself* with another geometry-value passed as an argument.
 - Every subclass of `GeometryValue` needs an `intersect` method, but these will be short. The argument is another geometry-value, but we do not know what kind. So we use double dispatch and call the appropriate method on the argument passing `self` to the method. For example, the `Point` class has an `intersect` method that calls `intersectPoint` with `self`.
 - So methods `intersectNope`, `intersectPoint`, `intersectLine`, `intersectVerticalLine`, and `intersectLineSegment` defined in each of our 5 subclasses of `GeometryValue` handle the 25 possible intersection combinations:
 - The 9 cases involving `Nope` are done for you. See the `GeometryValue` class—there is nothing more you need to do.
 - Next do the 9 remaining cases involving combinations that do not involve `LineSegment`. You will need to understand double-dispatch to avoid `is_a?` and `instance_of?`. As in the OCaml code, 3 of these 9 cases can just use one of the other cases because intersection is commutative.
 - What remains are the 7 cases where one value is a `LineSegment` and the other is not `Nope`. These cases are all “done” for you because all subclasses of `GeometryValue` inherit an `intersectLineSegment` method that will be correct for all of them. But it calls `intersectWithSegmentAsLineResult`, which you need to implement for each subclass of `GeometryValue`. Here is how this method should work:
 - * It takes one argument, which is a line segment. In OCaml, the corresponding variable was a `float * float * float * float`, but here it will actually be an instance of `LineSegment` and you can use the getter methods `x1`, `y1`, `x2`, and `y2` as needed.
 - * It assumes that `self` is the intersection of (1) some not-provided geometry-value, and (2) the line (vertical or not) containing the segment given as an argument.
 - * It returns the intersection of the not-provided geometry-value and the segment given as an argument.

Together the 5 `intersectWithSegmentAsLineResult` methods you write will implement the same algorithm as on lines 86–125 of the OCaml code.

And finally...

Which style do you prefer? Object-oriented (Ruby) or functional (OCaml)? Please note your preferences in the Brightspace during the assignment submission.

Good luck!